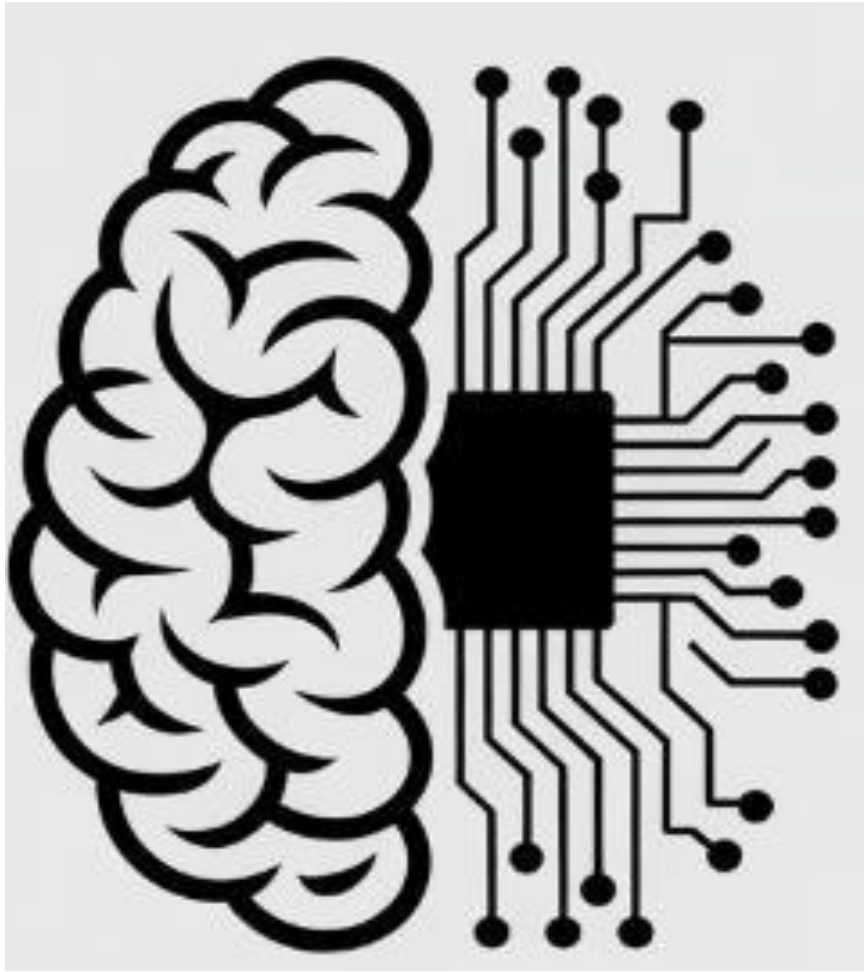


Neural Network Issues

Dr. Sobia Tariq Javed

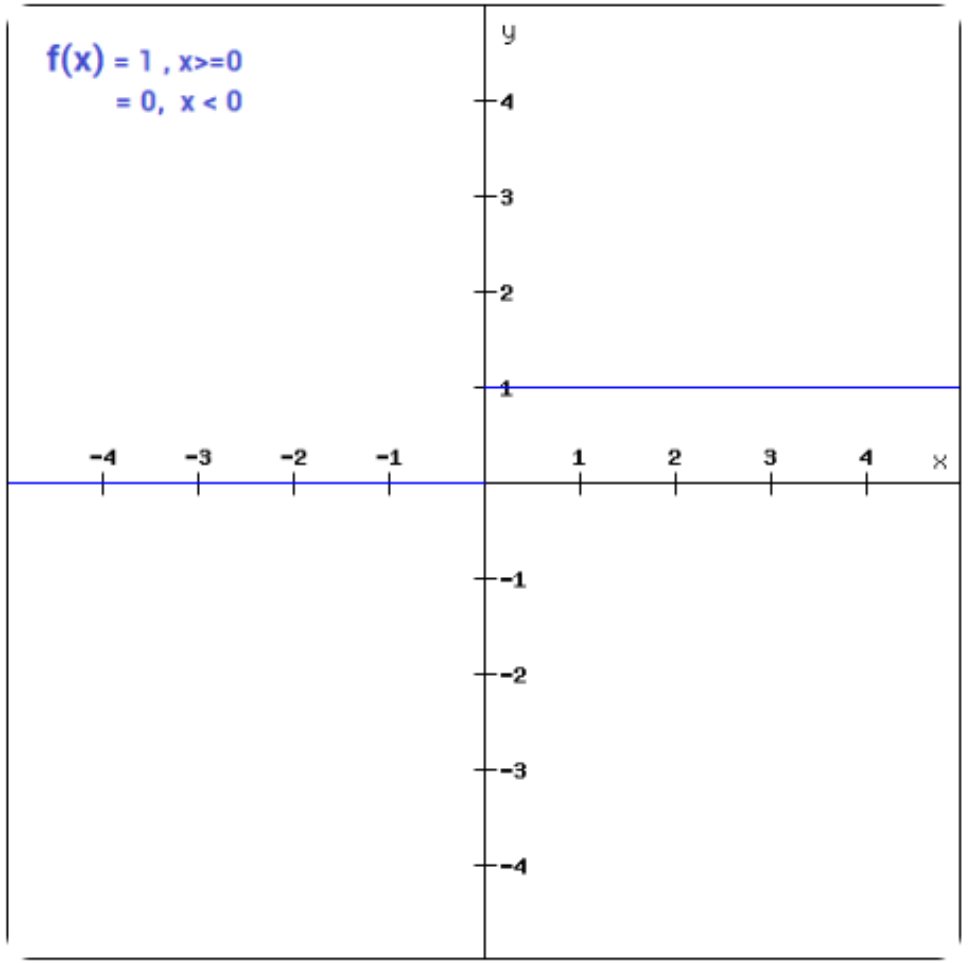
Activation Functions

Activation functions are essential in

a) **Data Preprocessing**
b) **When data is Non-linear**
c) **Learning complex patterns**

Types of Activation Functions

1) **Binary(Step) Function:**



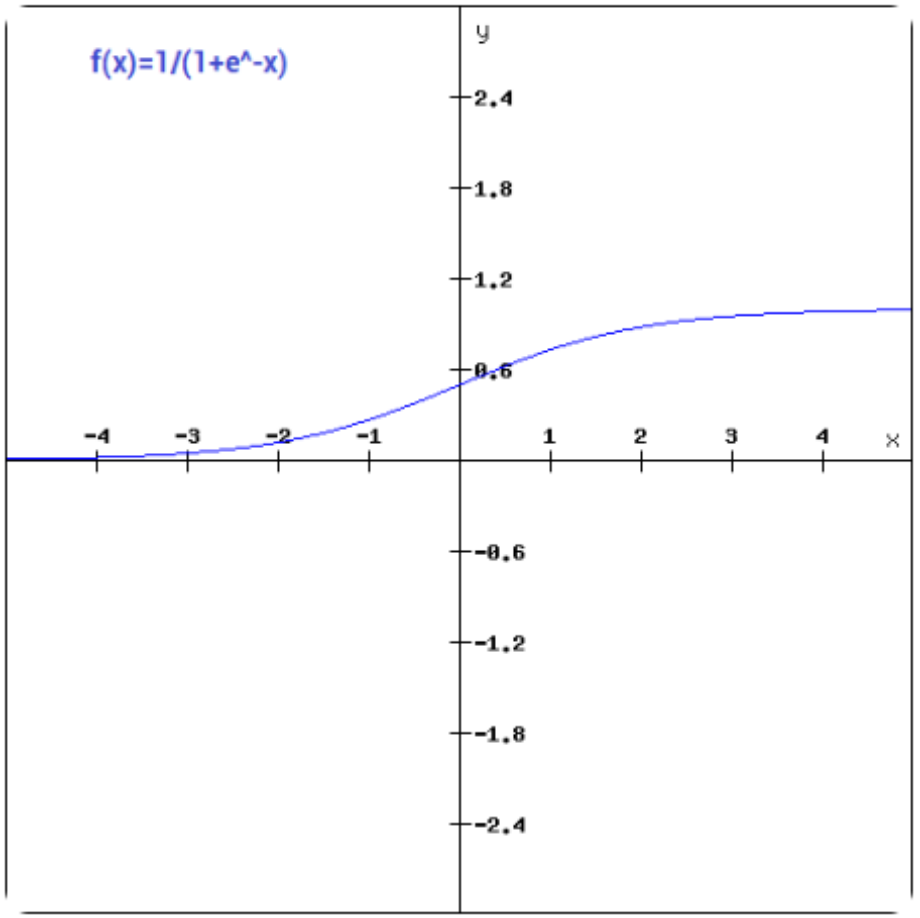
- Used in Binary Classification

Problem: Gradient becomes zero in backpropagation.

Types of Activation Functions

2) Sigmoid Function:

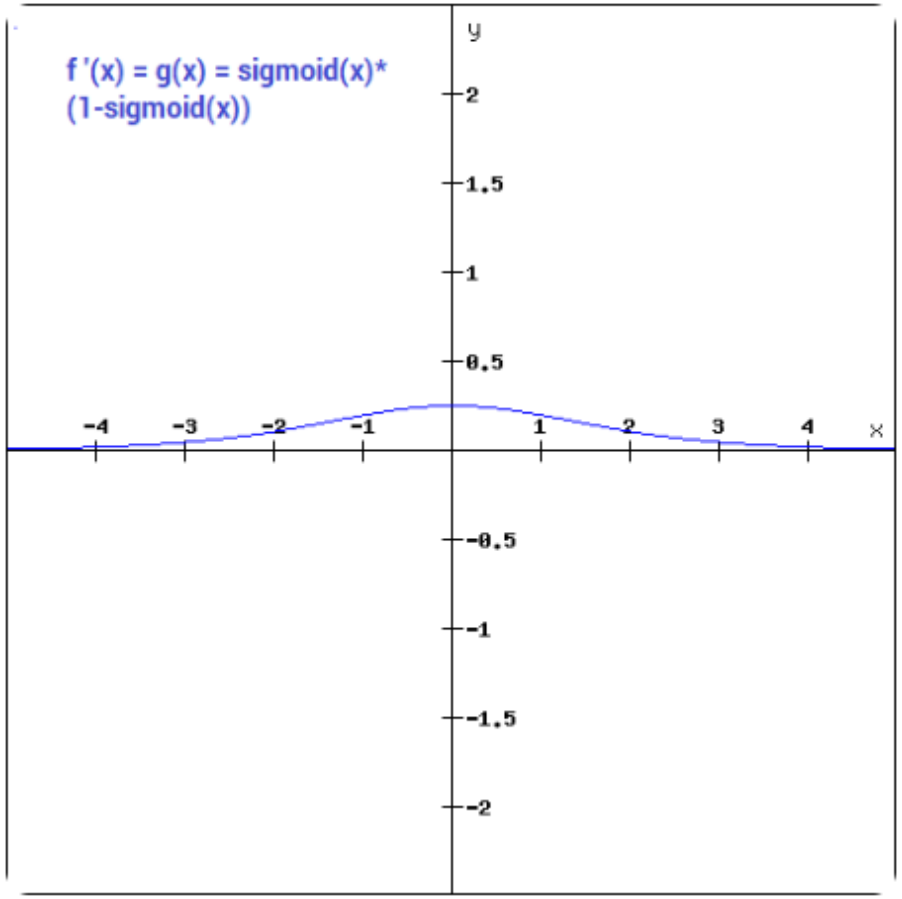
- When data is non-linear.
- Used in Binary Classification



Types of Activation Functions

2) Sigmoid Function:

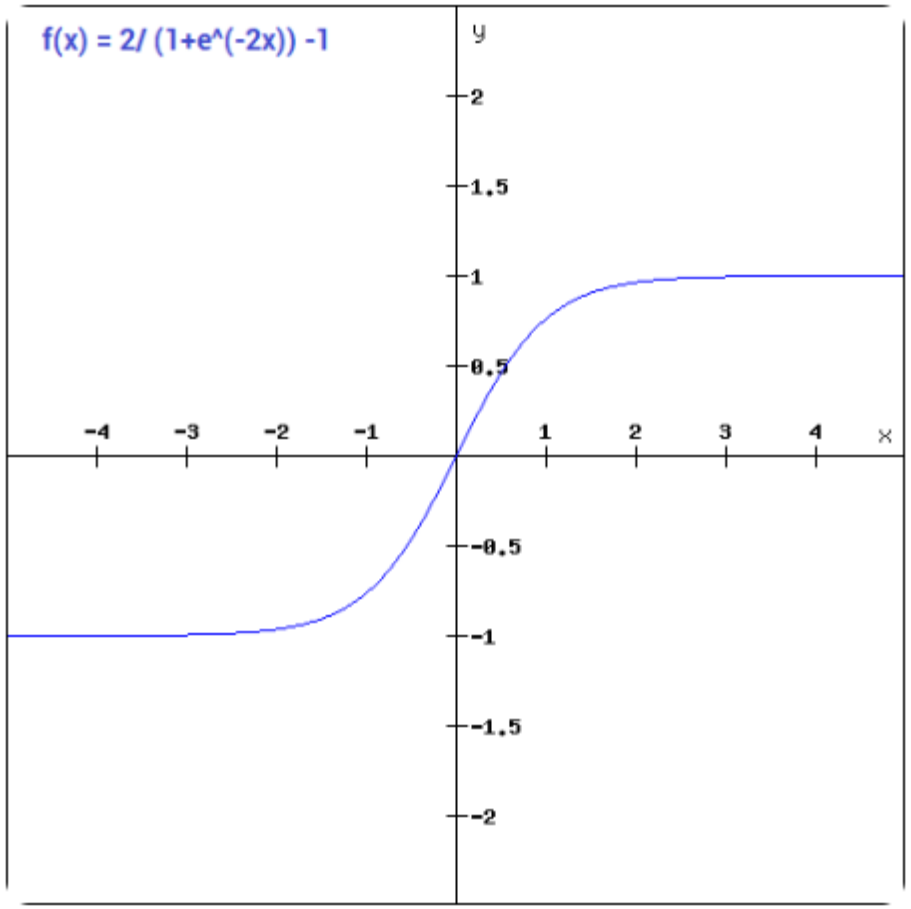
- Values outside -3 and 3 range will have small gradient.
- Sigmoid function is not symmetric around 0 (Unknown region)



Types of Activation Functions

3) Tanh Function:

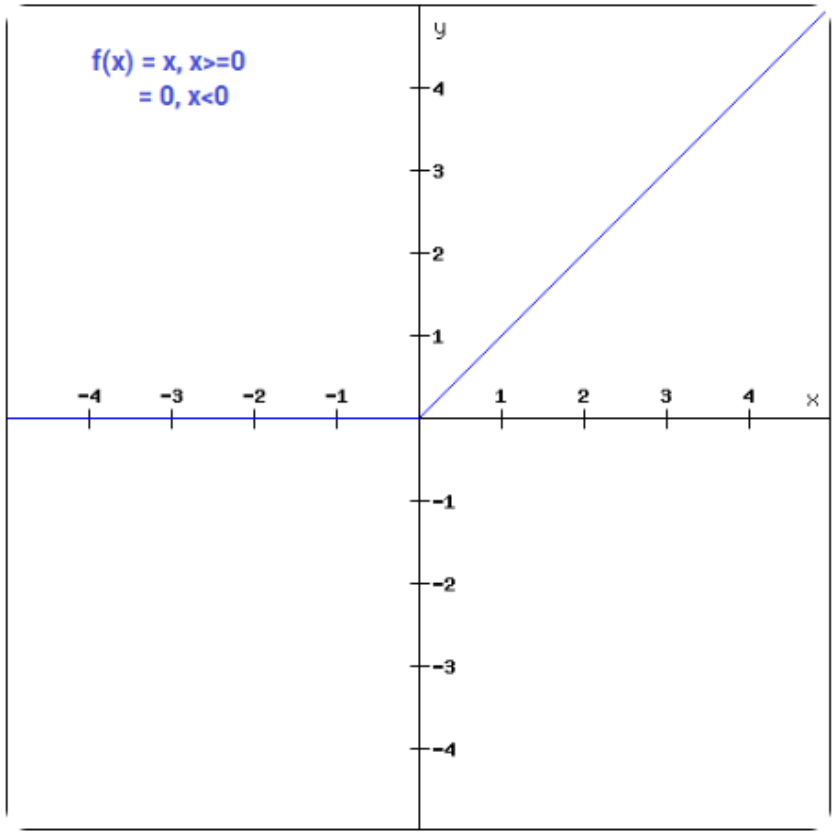
- Very similar to Sigmoid.
- Symmetric around 0.



Types of Activation Functions

3) ReLU Function:

- Non-Linear AF.
- ReLU stands for Rectified Linear Unit.
- Does not activate all neurons.
- Used for filter.

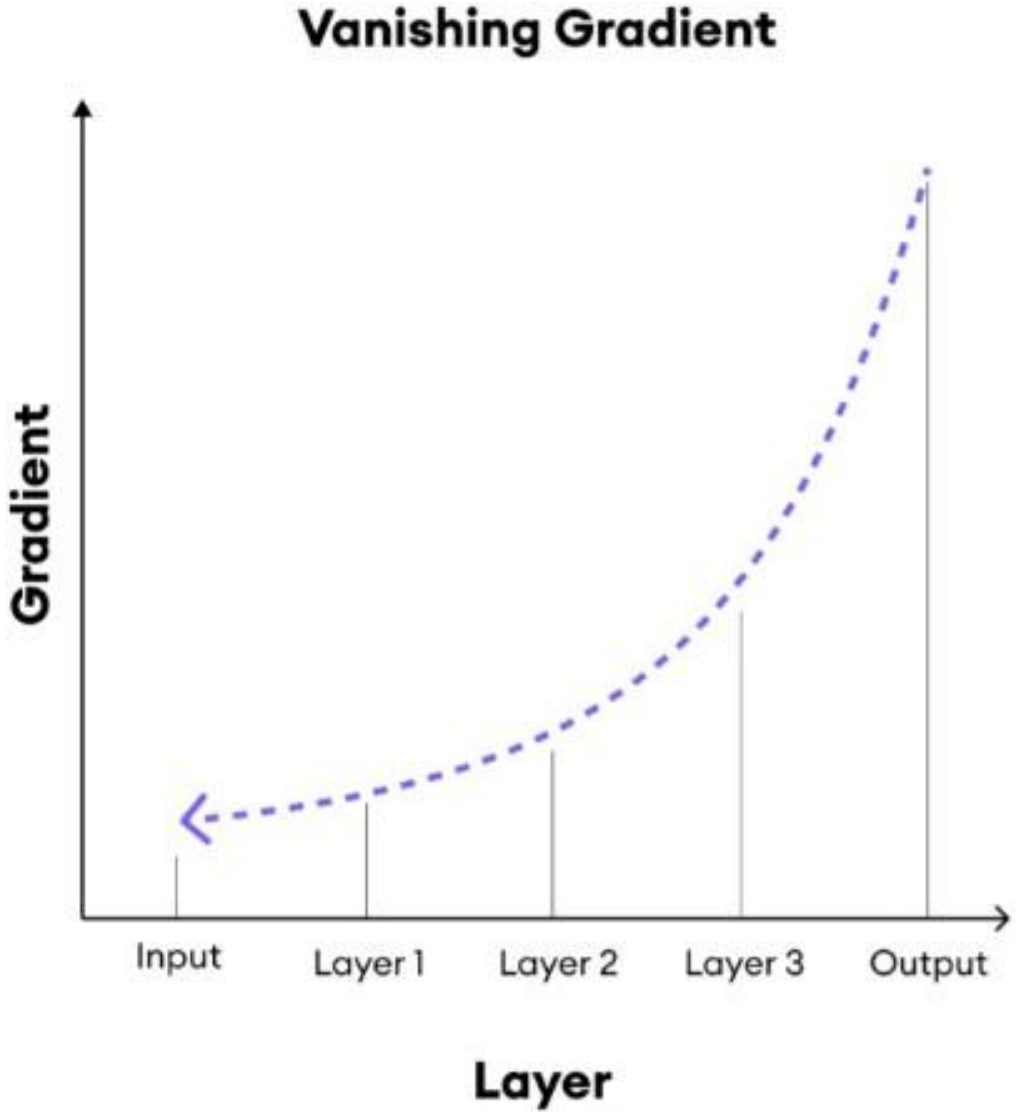


Challenges in Training Neural Network

- **1) Vanishing Gradient**
  - It is a problem where **gradients** become very small during **backpropagation**, preventing **deep neural networks** from learning effectively.
  - During **backpropagation**, gradients are multiplied across many layers.
  - If **gradients** are very **small** (less than 1), they keep **shrinking**.
  - Early layers receive **near-zero** updates.
  - The network **stops learning** effectively.

Challenges in Training Neural Network

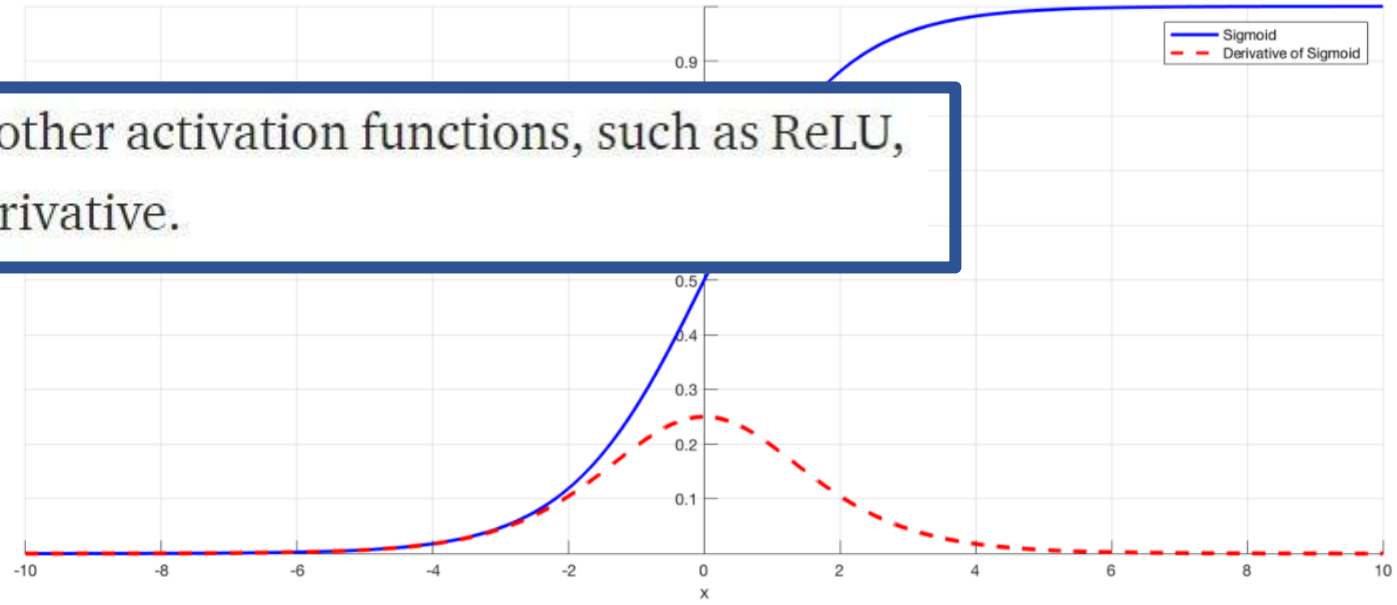
- 1) **Vanishing Gradient- Solution**
  - ReLU activation
  - Batch normalization
  - Proper weight initialization



Challenges in Training Neural Network

- The gradients of the loss function approaches zero, making the network hard to train.

The simplest solution is to use other activation functions, such as ReLU, which doesn't cause a small derivative.

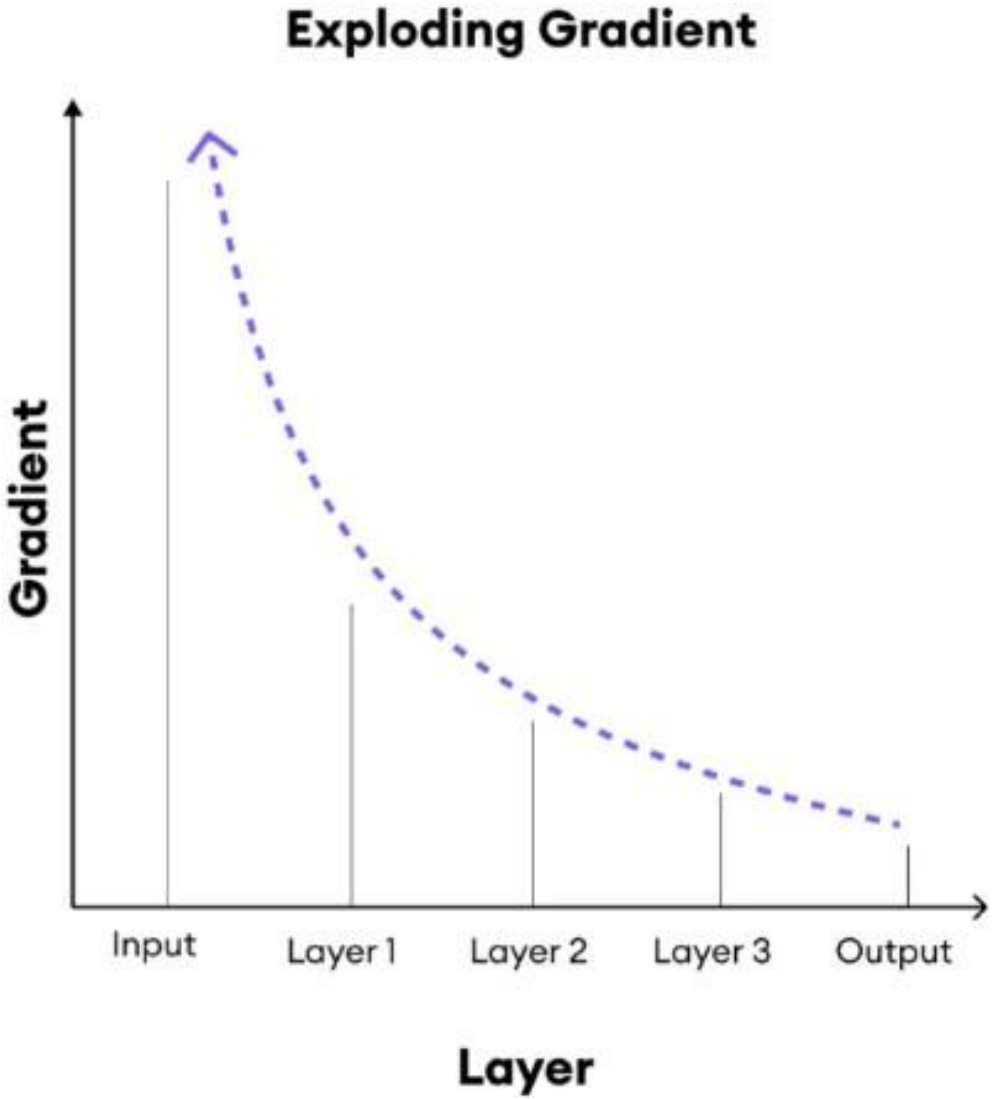


Challenges in Training Neural Network

- 2) **Exploding Gradient**
  - It is a problem where **gradients** grow too large during **back propagation**, causing unstable learning and large weight updates.
  - **Large gradients** cause very **large weight updates** during training.
  - **Weights** may become **unstable** and **grow uncontrollably.**
  - The model may produce **NaN values** or **infinite loss.**
  - **Training** becomes **unstable** and the network fails to **converge**.
  - This problem commonly occurs in **deep neural networks.**

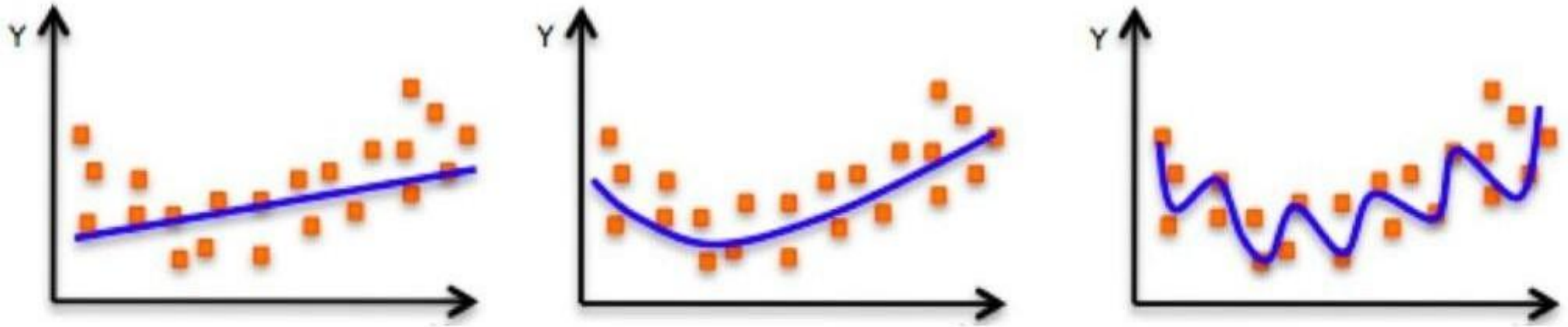
Challenges in Training Neural Network

- 2) **Exploding Gradient -** Solution
  - Gradient clipping
  - Proper weight initialization
  - Batch normalization
  - Smaller learning rate



Overfitting and Underfitting

- These describe how well a model generalizes from training data to unseen data.



Underfitting
Model does not have capacity to fully learn the data

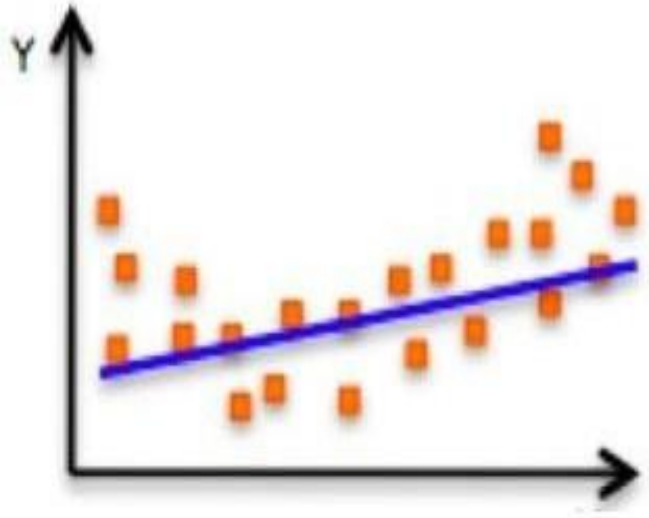
Ideal fit

Overfitting
Too complex, extra parameters, does not generalize well

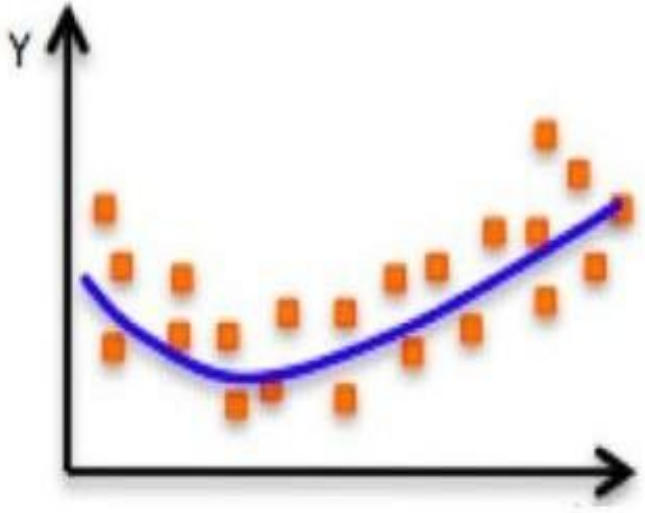
Underfitting

- It occurs when the **model is too simple** to capture the underlying pattern in the data.

- The model fails to learn even the **training data** properly.



Underfitting



Balanced

- **Characteristics**
  - Training error --- ⇧
  - Testing error --- ⇧
  - **Poor performance** everywhere

Underfitting

- **Bias**
  - It is the error caused by **oversimplified assumptions** in the model.
  - It measures how far the **model's predictions** are from the **true relationship**.

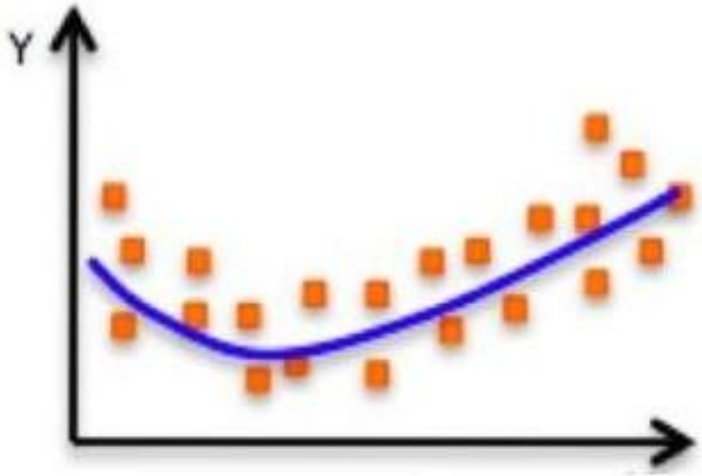
High Bias = Model is too simple

Solution:

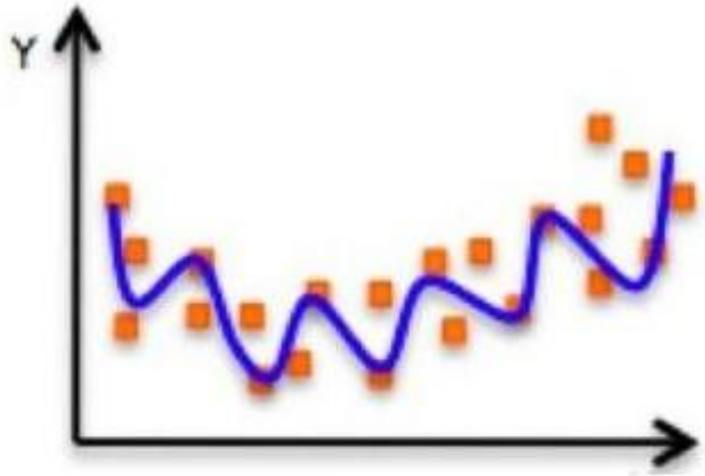
- Increase **model complexity**
- Train longer (**more epochs**)
- Reduce **regularization**
- Add better **features**

Overfitting

- Model memorizes training data instead of learning general patterns, leading to poor performance on new data.



Balanced

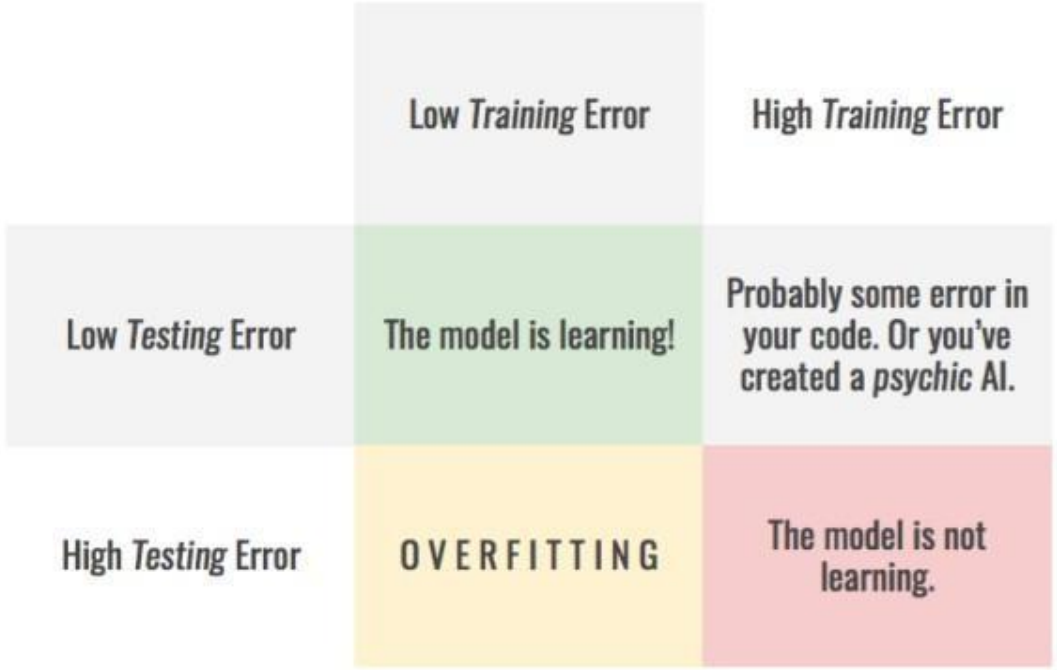


Overfitting

- **Characteristics**
  - Training error ---
  - Testing error ---
  - **Poor Generalization**

Overfitting

- To test this ability, a simple method consists in splitting the dataset into two parts: the training set and the test set.



Overfitting can be seen as the **difference** between the **training** and **testing** error.

Overfitting

- **Variance**
  - It is the error caused by **model sensitivity** to small changes in **training data**.
  - It measures how much **predictions change** if the **training data changes**.

High Variance = Model is too sensitive (overfits)

Solution:

- Collect more **Data**
- Simplify the **model**
- **Regularization**
- **Dropout** Regularization
- **Data Augmentation** and **noise**
- **Early stopping**

Bias Vs. Variance

Case	Bias	Variance	Result
Underfitting	High	Low	Poor learning
Good Fit	Balanced	Balanced	Good generalization
Overfitting	Low	High	Poor generalization

Bias – Variance Tradeoff

- Cannot minimize both simultaneously — improving one often worsens the other.

- Increase model complexity:
  - Bias ↓
  - Variance ↑
- Decrease complexity:
  - Bias ↑
  - Variance ↓

Simplify the Model

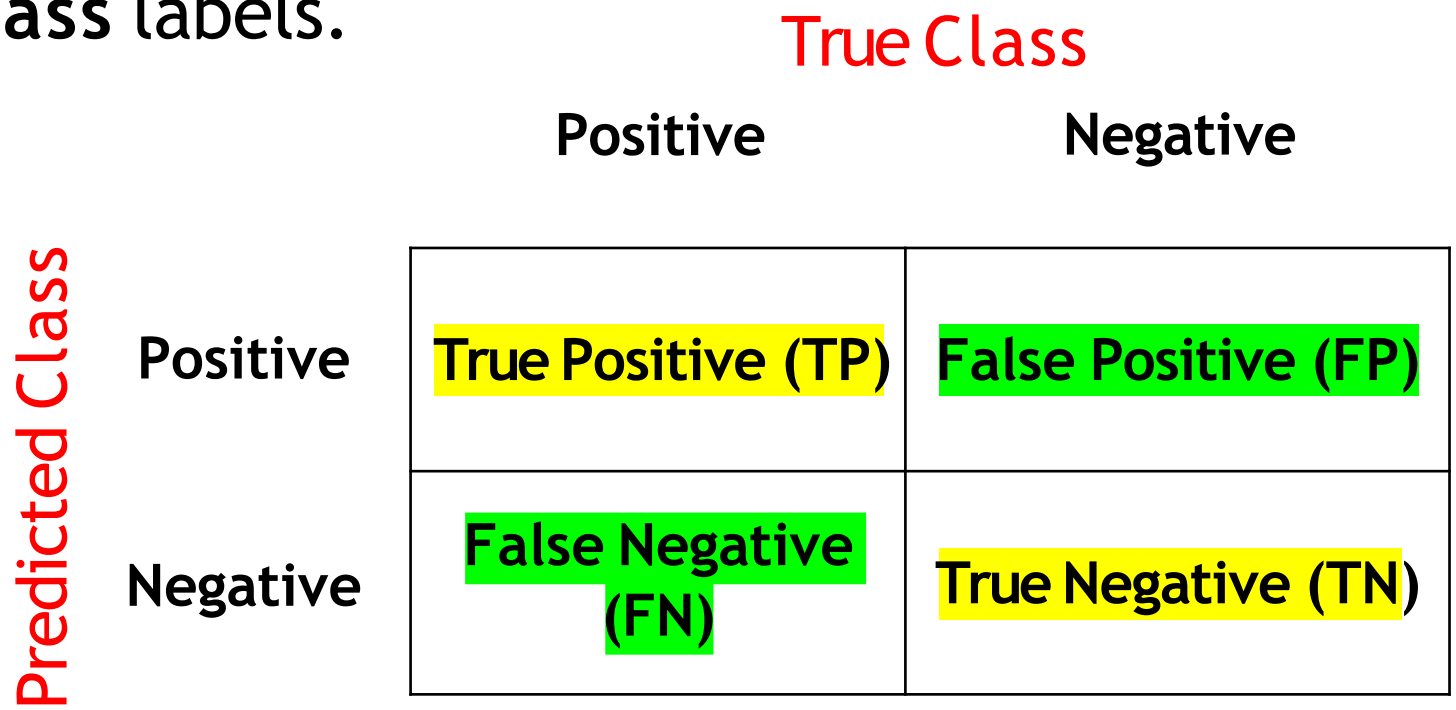
On the left, the model is too simple. On the right it overfits.

Reference

- Chapter 4 of Tom M. Mitchell “Machine Learning”

Confusion Matrix

- A **confusion matrix** is a NxN table used to evaluate the performance of a classification model by comparing **actual** and **predicted class** labels.



For a binary classification problem, we would have a 2 x 2 matrix as shown in figure with 4 values.

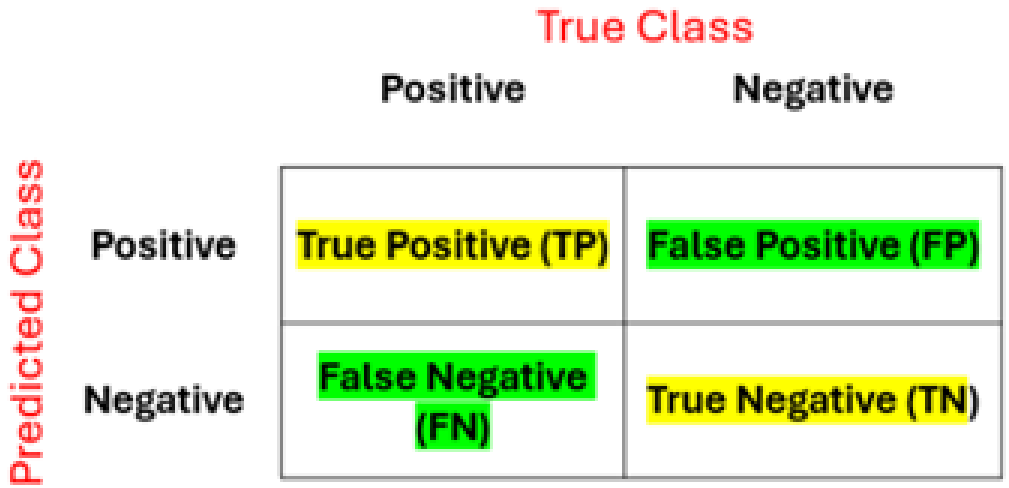
Evaluation: Confusion Matrix

TP - True Positive: Number of correct positive predictions that are actually positive.

FP - False Positive: Number of incorrect positive predictions that are actually negative.

FN - False Negative: Number of incorrect negative predictions that are actually positive

TN - True Negative: Number of correct negative predictions that are actually negative.



Evaluation Metric

1) Accuracy

- It measures how many **predictions** are **correct** out of total predictions.
- It shows overall performance of the model.
- Suitable when classes are balanced.

$$\text{Accuracy} = \frac{TP \quad +TN}{TP+FN+FP+TN} \qquad \text{Misclassification Error} = \frac{FP \quad +FN}{TP+FN+FP+TN}$$

- Overall correctness of the model.

Evaluation Metric

2) Precision

- Precision measures how many **predicted positive** cases are **actually positive**.
- It focuses on reducing **false positives**.

$$\text{Precision} = \frac{TP}{TP+FP}$$

- Reliability of **positive predictions**.

Evaluation Metric

3) Recall

- It measures how many **actual positive** cases are correctly detected.
- It focuses on reducing **false negatives**.

$$\text{Recall} = \frac{TP}{TP+FN}$$

- Ability to detect **positive cases**.

Evaluation Metric

4) F1-score

- It is the **harmonic mean** of **precision** and **recall.**
- It **balances** both **precision and recall.**
- Useful when dataset is **imbalanced.**
- F1 = 2 × (Precision × Recall) / (Precision + Recall)

$$F1 - \text{Score} = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$

- **Balanced performance** measure.

Evaluation Metric-Summary

1. **Accuracy:** Overall correct predictions
2. **Precision:** Correctness of positive predictions
3. **Recall:** Detection of actual positives
4. **F1-score:** Balance between precision and recall

Exercise

- A medical test is used to detect a disease. The following results were obtained:
  - 40 **patients actually** had the disease and were correctly **predicted as positive**
  - 10 patients **had the disease** but were **predicted negative**
  - 30 patients **did not have the disease** and were correctly **predicted negative**
  - 20 patients **did not have the disease** but were **predicted positive**
- **Construct the confusion matrix**
- Compute:
  - Accuracy
  - Precision
  - Recall
  - F1-score

TP = 40
FN = 10
TN = 30
FP = 20

Predicted Class \ True Class	Positive	Negative
Positive	40	20
Negative	10	30

Exercise

TP = 40
FN = 10
TN = 30
FP = 20

- **Accuracy**

$$Accuracy = \frac{TP + TN}{Total}$$

$$= \frac{40 + 30}{100} = 0.70$$

Model is reasonably accurate but not highly reliable.

- Accuracy = 70%

$$\text{Misclassification Error} = \frac{FP \quad + FN}{TP + FN + FP + TN}$$

Misclassification Error = 1 – accuracy
= 1 – 0.70 = 0.30 or 30%

Predicted Class	True Class: Positive	True Class: Negative
Positive	40	20
Negative	10	30

Exercise

TP = 40
FN = 10
TN = 30
FP = 20

- **Precision**

Precision is moderate and should be improved.

$$Precision = \frac{TP}{TP + FP}$$

$$= \frac{40}{40 + 20} = 0.67$$

- Precision = 66.7%

Predicted Class	True Class: Positive	True Class: Negative
Positive	40	20
Negative	10	30

Exercise

TP = 40
FN = 10
TN = 30
FP = 20

- **Recall**

Recall is strong and desirable.

$$Recall = \frac{TP}{TP + FN}$$

$$= \frac{40}{40 + 10} = 0.80$$

- Recall = 80%

True Class

Predicted Class	Positive	Negative
Positive	40	20
Negative	10	30

Exercise

TP = 40
FN = 10
TN = 30
FP = 20

- **F1-score**

Balanced but slightly biased toward recall.

$$F1 = \frac{2PR}{P+R}$$

$$= \frac{2(0.67)(0.80)}{0.67+0.80} = 0.73$$

- F1-score = 73%

True Class

Predicted Class	Positive	Negative
Positive	40	20
Negative	10	30

Analysis

$$Accuracy = \frac{TP + TN}{Total}$$

For Imbalanced Dataset: Classification Accuracy is not a correct measure

- **1. Accuracy = 70%**
  - **What it tells us:**
    - Accuracy shows the **overall correctness** of the model.
    - It answers: **"Out of all patients, how many were correctly classified?"**
  - **Analysis:**
    - The model correctly **predicts 70 out of 100 patients**.
    - While this seems reasonable, accuracy alone can be misleading, especially in medical problems.
    - It does not distinguish between types of errors (false positives vs false negatives).

Analysis

$$Precision = \frac{TP}{TP + FP}$$

Do not rely on precision when recall is critical

- **2. Precision = 66.7%**
  - **What it tells us:**
    - Precision measures how **reliable positive predictions** are.
    - It answers: “When the model says a patient has the disease, how often is it correct?”
  - **Analysis:**
    - Out of all predicted positive cases (60), only 40 are truly diseased.
    - About 1 in 3 positive predictions is wrong (false alarms).
    - This indicates moderate reliability of positive predictions.

Analysis

$$Recall = \frac{TP}{TP + FN}$$

Avoid recall alone when false positives matter

- **3. Recall (Sensitivity) = 80%**
  - **What it tells us:**
    - Recall measures the model's ability to detect actual disease cases.
    - It answers: "Out of all patients who truly have the disease, how many did we correctly identify?"
  - **Analysis:**
    - The model correctly identifies 80% of diseased patients.
    - Only 20% (10 patients) are missed.
    - This is quite good, especially in medical diagnosis where missing a disease is risky.

Analysis

$$F1 - \text{Score} = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$

Avoid F1 when both classes (positive s negative) are equally important

- **3. 4. F1-Score = 72.7%**
  - **What it tells us:**
    - F1-score is the balance between precision and recall.
    - It is useful when we want a single measure of performance, especially with uneven errors.
  - **Analysis:**
    - The value (≈73%) shows a moderate balance:
    - Good recall (80%)
    - Lower precision (66.7%)
    - Indicates the model is better at detecting disease than avoiding false alarms.

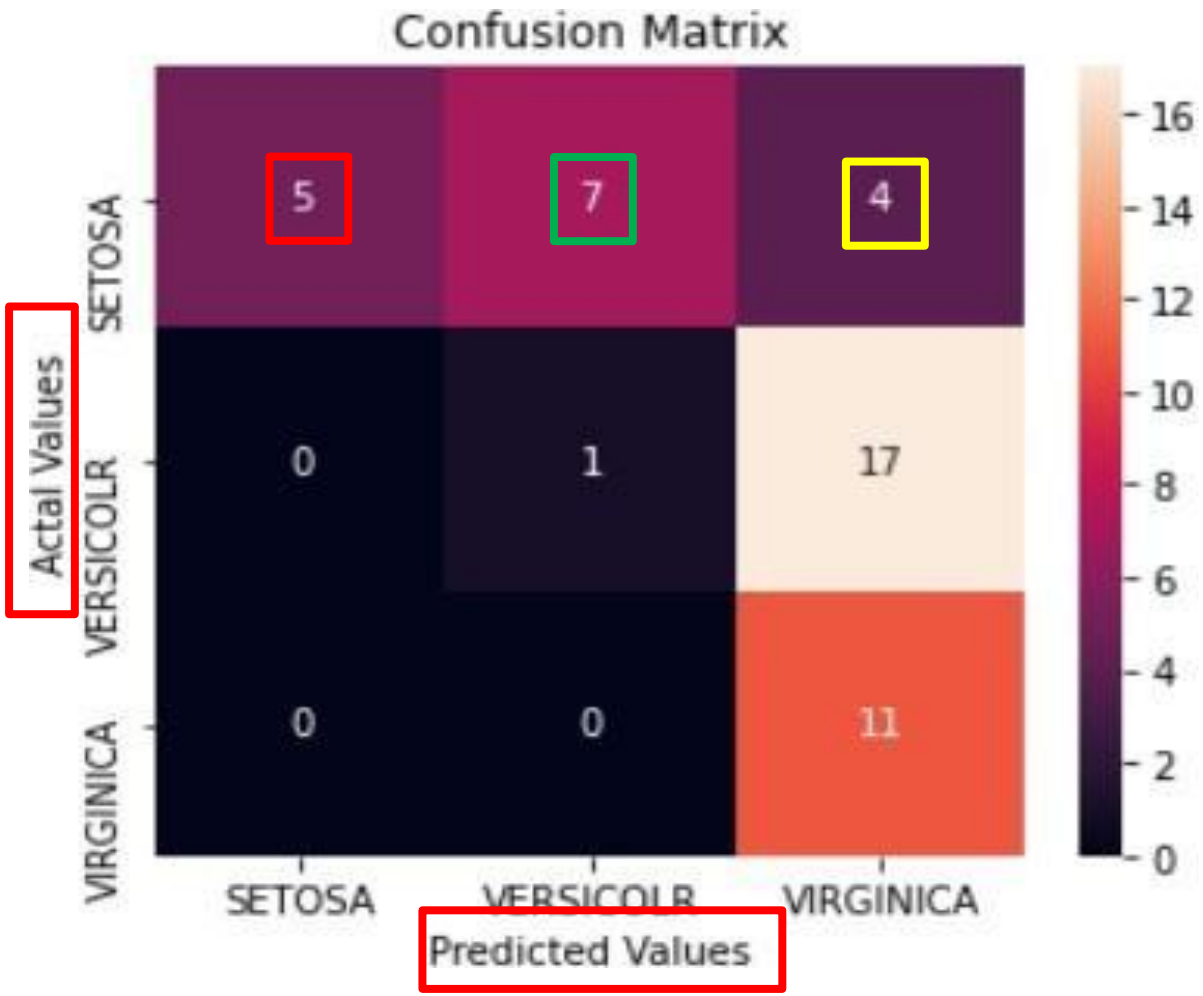
Confusion Matrix: Multiclass

- **IRIS Dataset**
  - The dataset has 3 flowers as outputs or classes,
    - **Versicolor**
    - **Virginica**
    - **Setosa.**

  - How to calculate TP, FP, TN, FN

Confusion Matrix: Multiclass

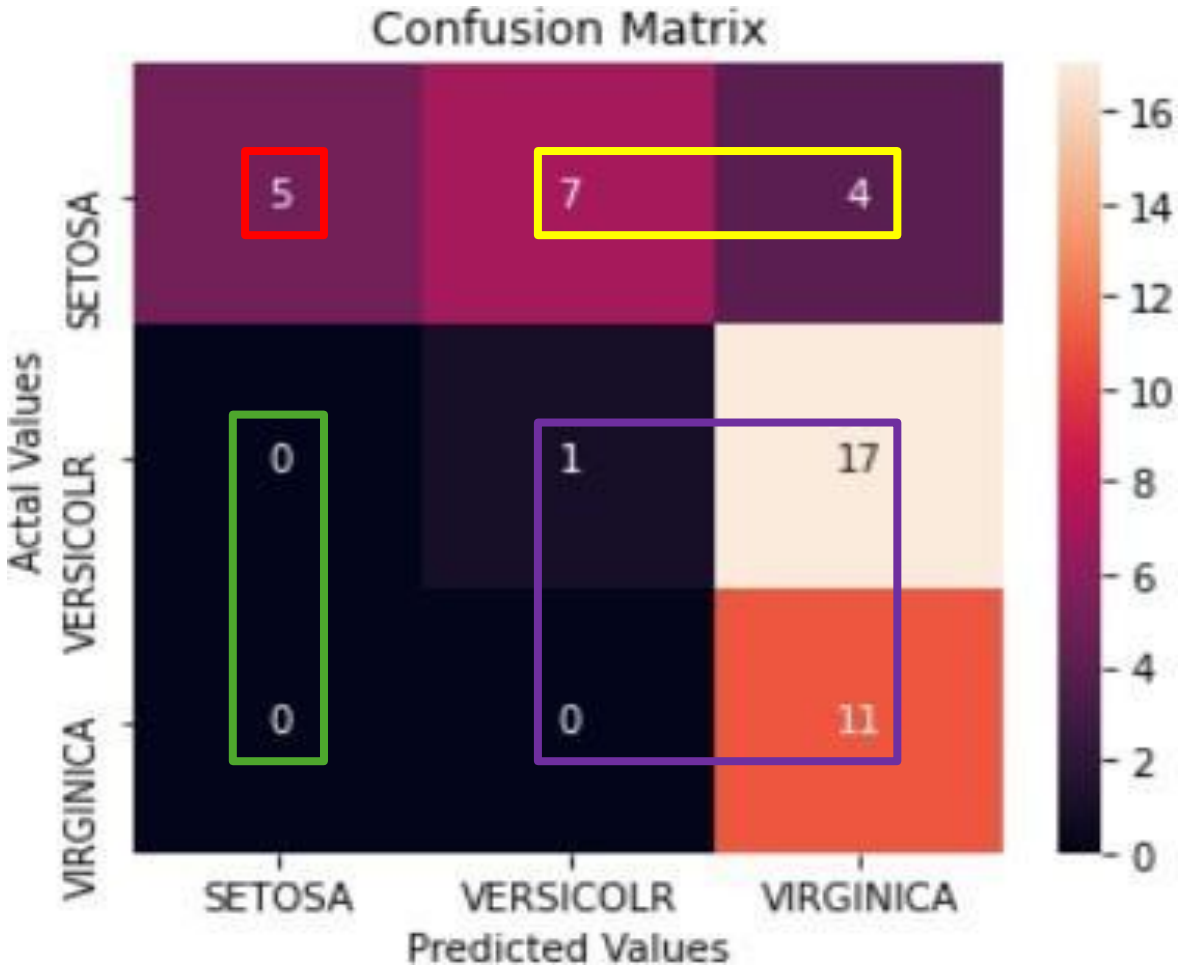
- What it tells?
- Actual
  - Setosa 5+7+4 = 16
  - Versicolr = 0+1+17 = 18
  - Virginica = 0+0+11 = 11
- Predicted
  - Out of the 16 Setosa
    - 5 are predicted correct
    - 7 are predicted as vericolr
    - 4 are predicted as virginica



Confusion Matrix: Multiclass

- How to calculate FN, FP, TN, TP?
- SETOSA:
  - TP: 5
  - FN: 7 + 4
  - FP: 0 + 0
  - TN: 1 + 17 + 0 + 11

H.W: compute for others as well



Confusion Matrix: Multiclass

- **How to calculate FN, FP, TN, TP :**
- **FN:** The False-negative value for a class will be the sum of values of corresponding rows except for the TP value.
- **FP:** The False-positive value for a class will be the sum of values of the corresponding column except for the TP value.
- **TN:** The True Negative value for a class will be the sum of values of all columns and rows except the values of that class that we are calculating the values for.
- **TP:** The True positive value is where the actual value and predicted value are the same.

Confusion Matrix: Multiclass

- **MNIST Dataset: labels[0-G]**

```
[[5825    1   37    9   12   20   31   13   12   22]
 [   0 6657   53   23   14   34   25   46  101   17]
 [   1   19 5627   44   12   13    6   52   12    3]
 [   4   21   50 5827    0   67    2   24  111   59]
 [   6   12   48    6 5480   33    8   40   33   61]
 [  10    7    9   79    1 5095   38    2   13    8]
 [  26    1   30   14   43   45 5789    4   20    2]
 [   1    4   29   34    6    1    0 5872    3   26]
 [  38   12   63   51   10   51   19   11 5447   24]
 [  12    8   12   44  264   62    0  201   99 5727]]
```

Micro and Macro Averaging (in classification evaluation)

- When you have multi-class classification, metrics like **precision, recall, and F1-score** can be computed in different ways. The two most important strategies are:
  - Macro averaging
  - Micro averaging

1. Macro Averaging (Class-wise Equal Importance)

- Compute the metric individually for each class, then take the simple average.

For each class i:

1. Compute Precision_i, Recall_i, F1_i
2. Then average:

$$\text{Macro Precision} = \frac{1}{K}\sum_{i=1}^{K}\text{Precision}_i$$

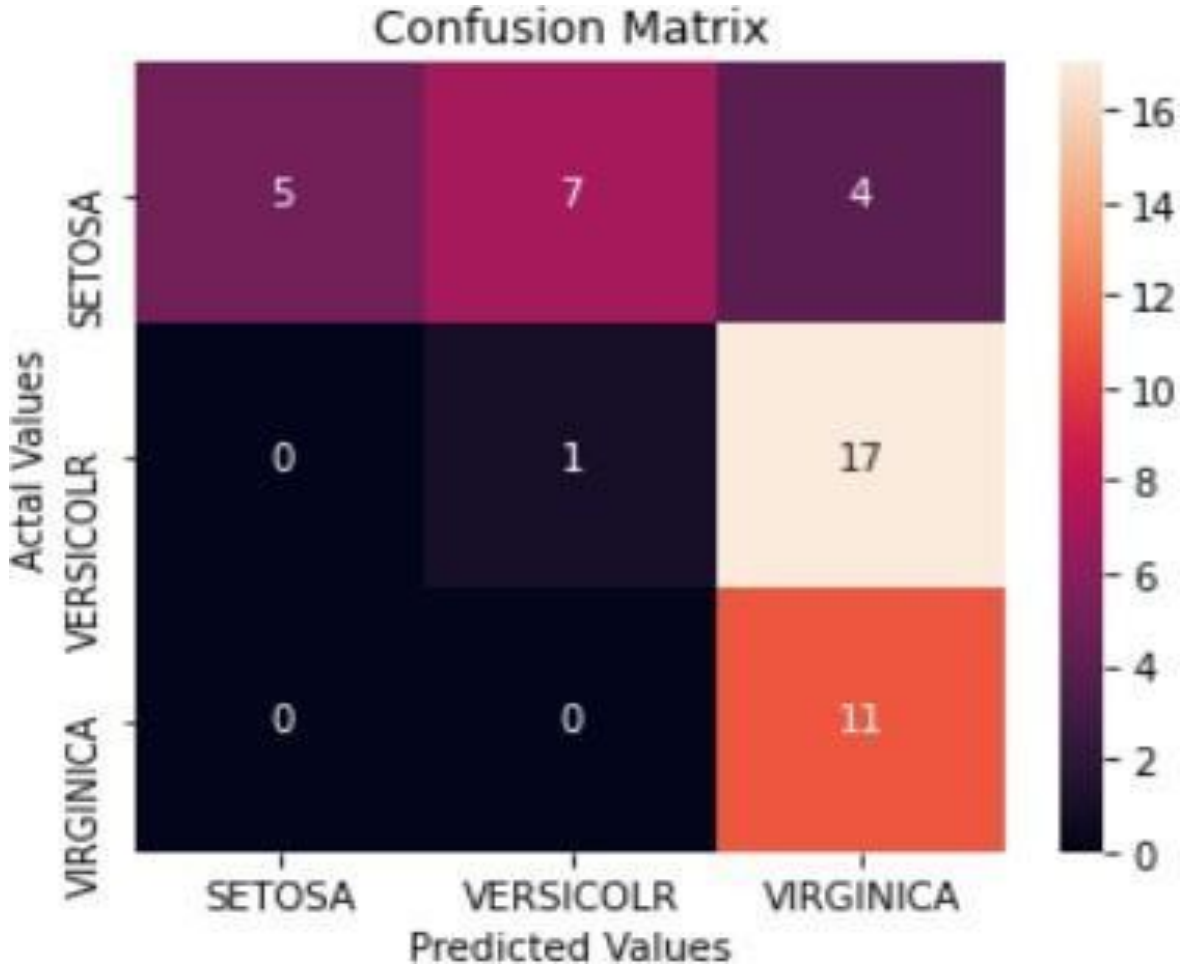
(where K = number of classes)

When to use
- Classes are imbalanced
- You care about minority class performance
- You want a fair evaluation across all classes

If:
Class A = very large (95% data)
Class B = very small (5% data)
Macro ensures Class B is **not ignored**

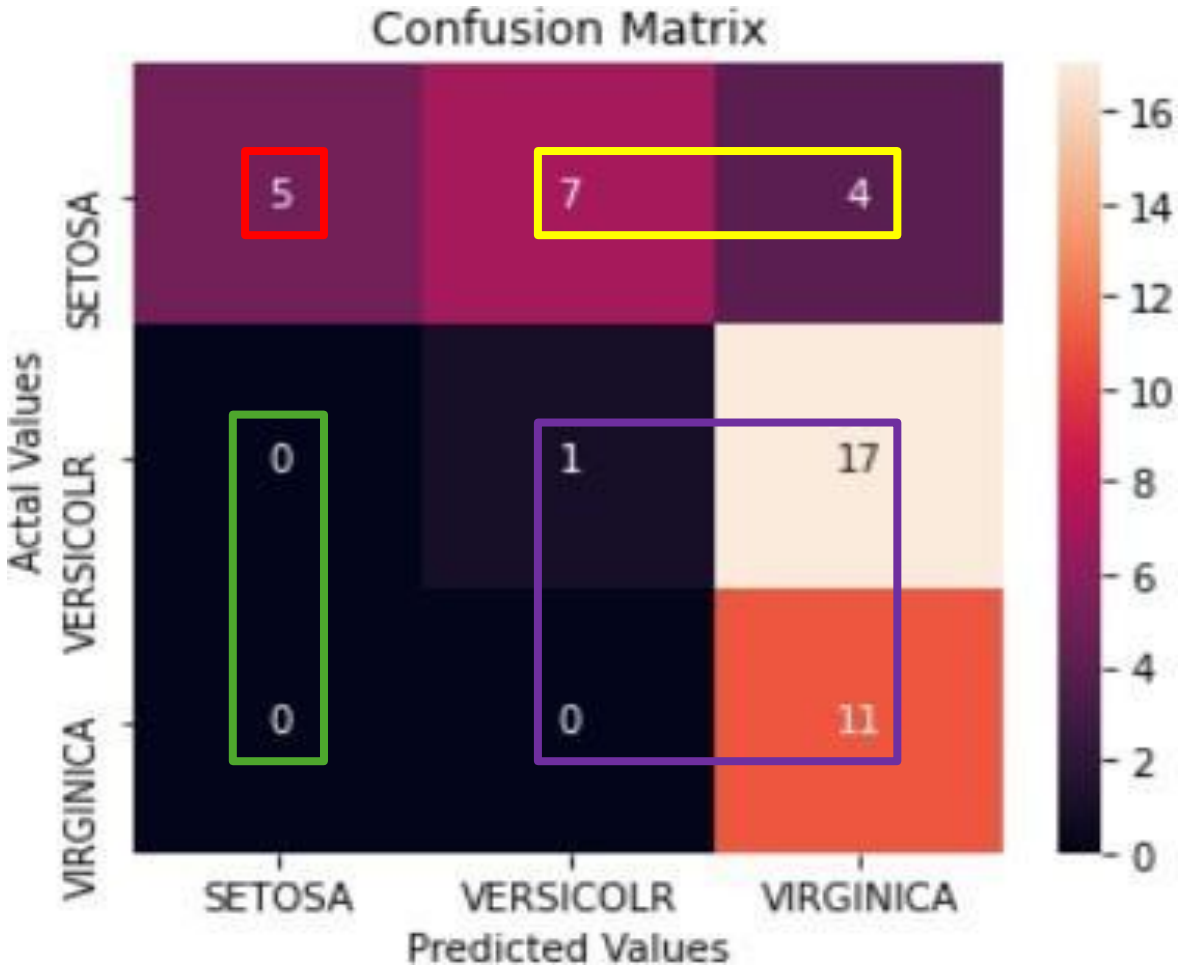
Macro Averaging

- Step 1: Total samples
  - N = 5+7+4+0+1+17+0+0+11 = 45
- Step 2: Compute
  - TP, FP, FN per class



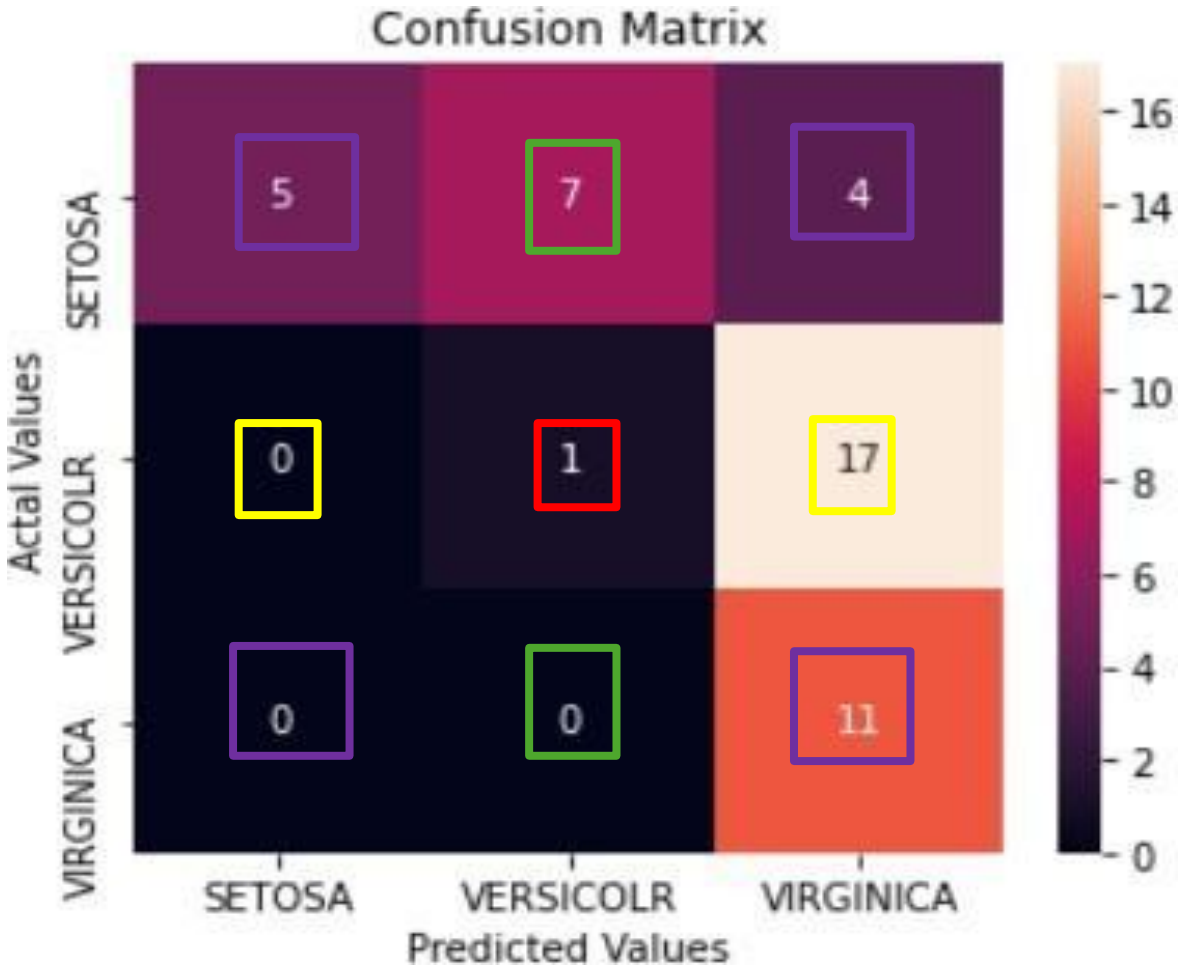
Macro Averaging

- SETOSA:
  - TP: 5
  - FN: 7 + 4
  - FP: 0 + 0
  - TN: 1 + 17 + 0 + 11



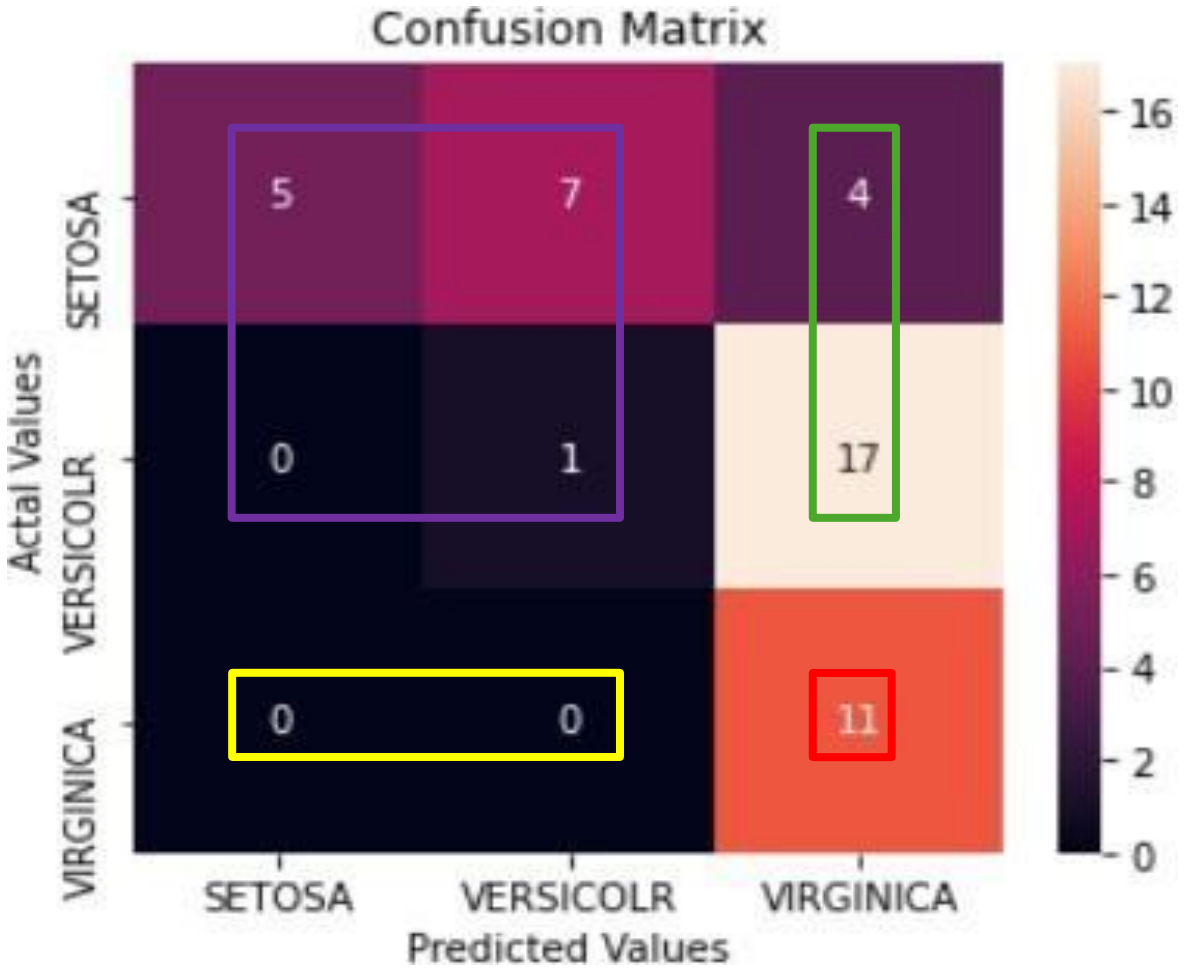
Macro Averaging

- VERSICOLR:
  - TP: 1
  - FN: 0 + 17
  - FP: 7 + 0
  - TN: 5 + 4 + 0 + 11



Macro Averaging

- **VIRGINICA:**
  - **TP:** 11
  - **FN:** 0 + 0
  - **FP:** 17 + 4
  - **TN:** 5 + 7 + 0 + 1



Macro Averaging

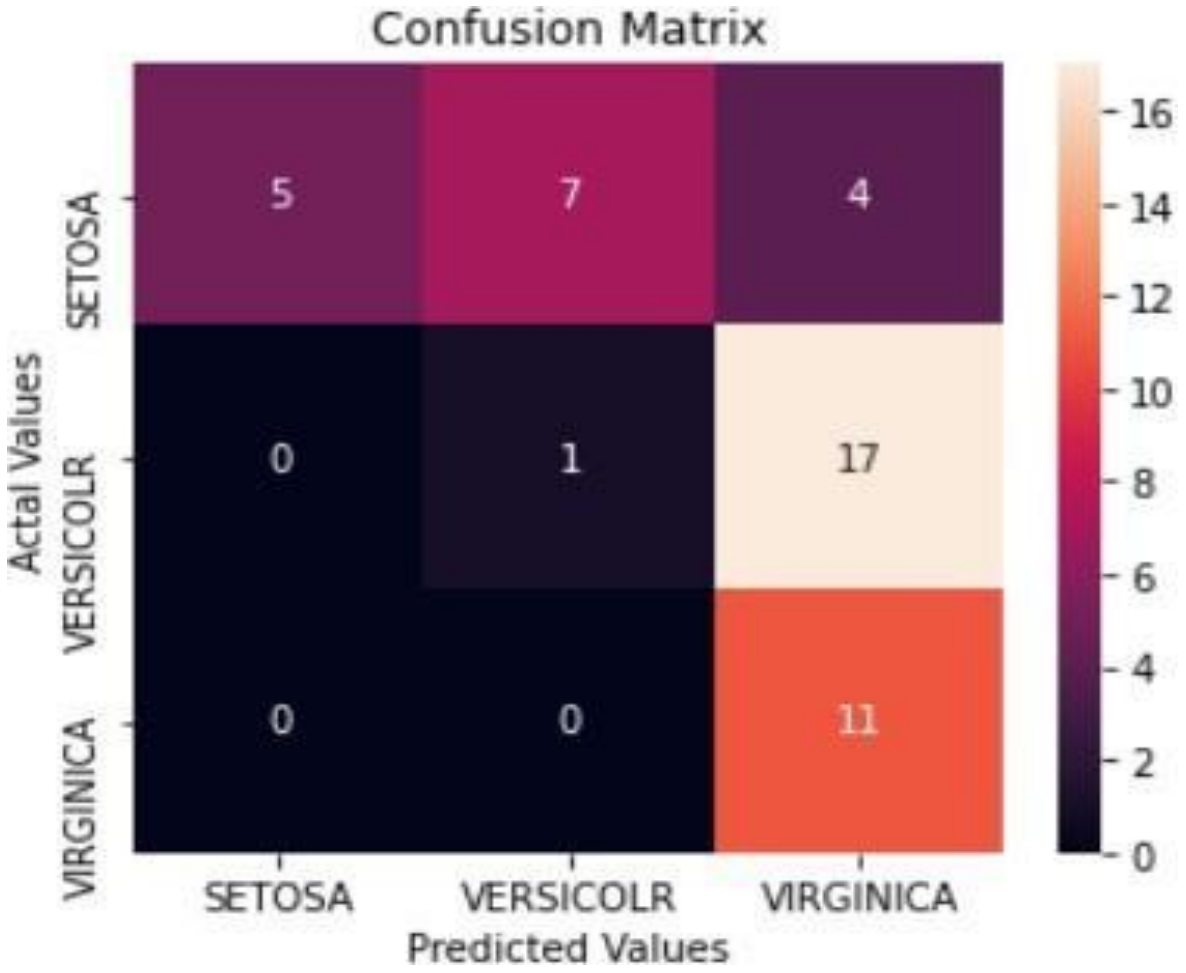
$$\text{Precision} = \frac{TP}{TP+FP}$$

- **Precision**
- **P(SETOSA) = $\frac{5}{5+0}$ =1**
- **P(VERSICOLOR) = $\frac{1}{1+7}$ =0.125**
- **P(VIRGINICA) = $\frac{11}{11+21}$ =0.344**

Macro Precision

$$\frac{1 + 0.125 + 0.344}{3} = 0.4897$$

- **SETOSA:**
  - **TP: 5**
  - **FN: 7 + 4**
  - **FP: 0 + 0**
  - **TN: 1 + 17 + 0 + 11**
- **VERSICOLR:**
  - **TP: 1**
  - **FN: 0 + 17**
  - **FP: 7 + 0**
  - **TN: 5 + 4 + 0 + 11**
- **VIRGINICA:**
  - **TP: 11**
  - **FN: 0 + 0**
  - **FP: 17 + 4**
  - **TN: 5 + 7 + 0 + 1**



2. Micro Averaging (Global Performance)

- Combine contributions of all classes first, then compute the metric globally.

1. Sum all:
   - True Positives (TP)
   - False Positives (FP)
   - False Negatives (FN)
2. Then compute:

$$\text{Micro Precision} = \frac{\sum TP}{\sum TP + \sum FP}$$

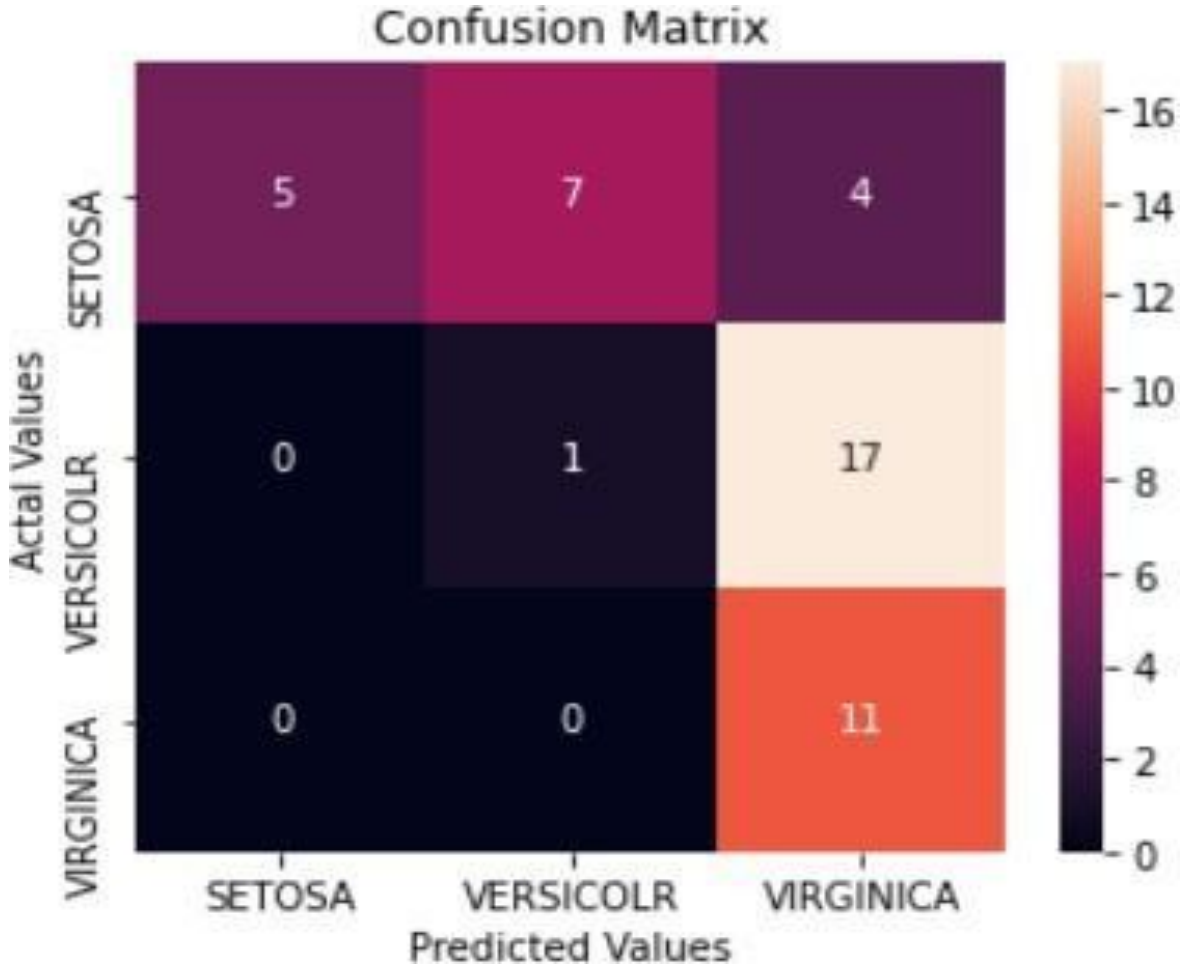
$$\text{Micro Recall} = \frac{\sum TP}{\sum TP + \sum FN}$$

Use it when:

- Dataset is balanced or naturally distributed
- You care about overall system accuracy
- You want performance at the instance level

Micro Averaging

- Step 1: Total samples
  - N = 5+7+4+0+1+17+0+0+11 = 45
- Step 2: Compute Total
  - TP, FP, FN



$$\text{Precision} = \frac{TP}{TP+FP}$$

Micro Averaging

- **SETOSA:**
  - TP: 5
  - FN: 7 + 4
  - FP: 0 + 0
  - TN: 1 + 17 + 0 + 11
- **VERSICOLR:**
  - TP: 1
  - FN: 0 + 17
  - FP: 7 + 0
  - TN: 5 + 4 + 0 + 11
- **VIRGINICA:**
  - TP: 11
  - FN: 0 + 0
  - FP: 17 + 4
  - TN: 5 + 7 + 0 + 1

Total TP

$$TP = 5 + 1 + 11 = 17$$

Total FP

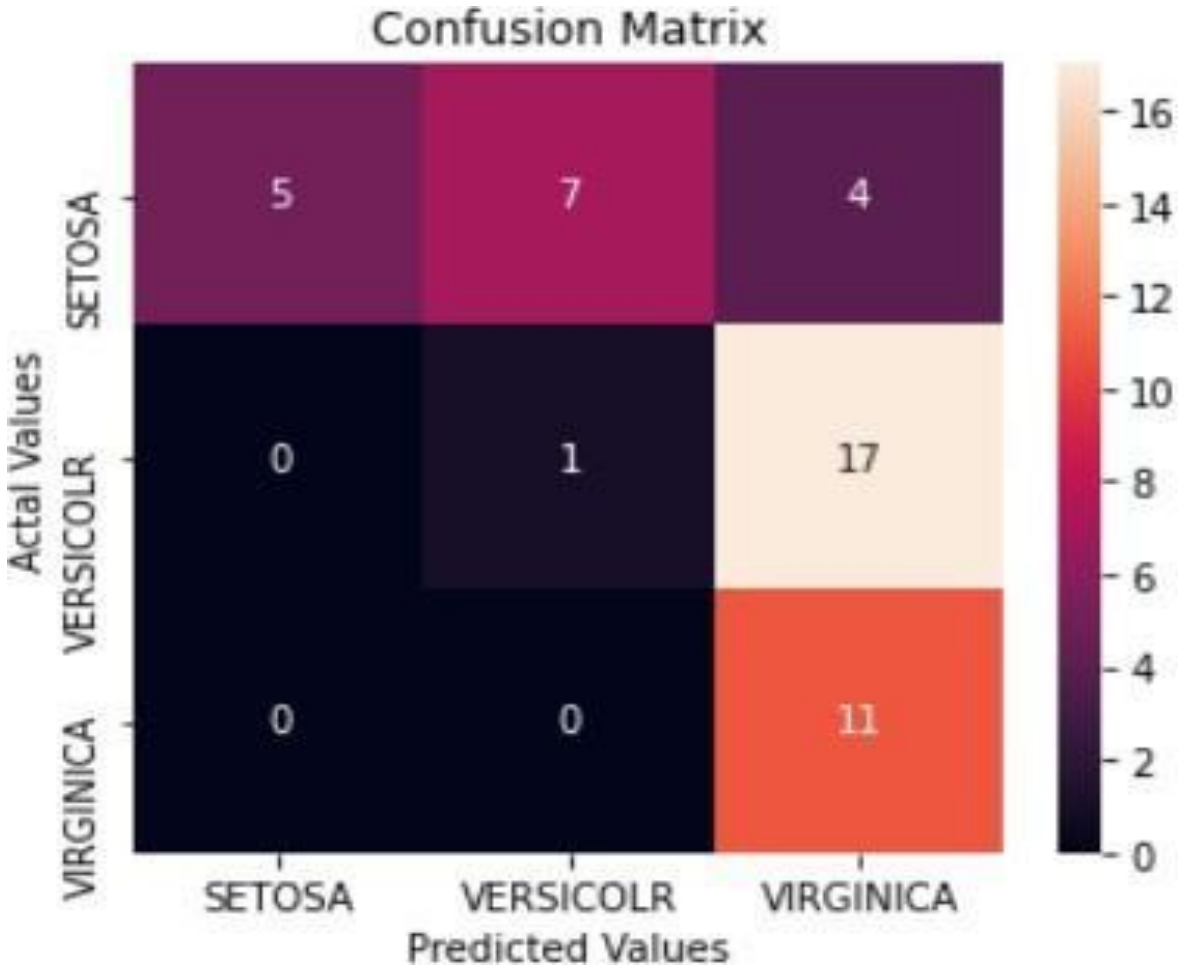
$$FP = 0 + 7 + 21 = 28$$

Total FN

$$FN = 11 + 17 + 0 = 28$$

Micro Recall

$$\text{Micro Recall} = \frac{17}{17 + 28} = \frac{17}{45} \approx 0.378$$



Micro and Macro Averaging (in classification evaluation)

- Macro = “How well does the model perform on each class fairly?”
- Micro = “How many predictions are correct overall?”